Detecting and Evaluating Order-Dependent Flaky Tests in JavaScript

Negar Hashemi*, Amjed Tahir*, Shawn Rasheed†, August Shi‡, Rachel Blagojevic*
*Massey University, New Zealand
{n.hashemi;a.tahir;r.v.blagojevic}@massey.ac.nz
†Universal College of Learning, New Zealand
s.rasheed@ucol.co.nz
‡ The University of Texas at Austin, USA
august@utexas.edu

***Abstract*—Flaky tests pose a significant issue for software testing. A test with a non-deterministic outcome may undermine the reliability of the testing process, making tests untrustworthy. Previous research has identified test order dependency as one of the most prevalent causes of flakiness, particularly in Java and Python. However, little is known about test order dependency in JavaScript tests. This paper aims to investigate test order dependency in JavaScript projects that use Jest, a widely used JavaScript testing framework. We implemented a systematic approach to randomise tests, test suites and `describe` blocks and produced 10 unique test reorders for each level. We reran each order 10 times (100 reruns for each test suite/project) and recorded any changes in test outcomes. We then manually analysed each case that showed flaky outcomes to determine the cause of flakiness. We examined our detection approach on a dataset of 81 projects obtained from GitHub. Our results revealed 55 order-dependent tests across 10 projects. Most order-dependent tests (52) occurred between tests, while the remaining three occurred between `describe` blocks. Those order-dependent tests are caused by either *shared files* (13) or *shared mocking state* (42) between tests. While sharing files is a known cause of order-dependent tests in other languages, our results underline a new cause (shared mocking state) that was not reported previously.**

***Index Terms*—Flaky Tests, Test Order Dependency, JavaScript, Jest**

I. Introduction

Flaky tests (i.e., tests with non-deterministic outcomes) may pass on some runs and fail on others on the same version of the code. Flaky tests are known to negatively impact product quality and delivery [1]–[3], as well as testing-related activities [4], [5]. A flaky test that fails may not indicate any bug in the code, and developers waste time trying to identify non-existent bugs. Flaky tests can then lead developers to lose trust in their builds and ignore test failures [6], resulting in additional costs for software projects. Indeed, flaky tests present a significant challenge in the software industry [1], [3], [7], [8]. A report from Microsoft noted that flaky tests could cost up to $7.2 million per year for Microsoft Dynamics product line [9]. GitHub also reported that 9% of their commits had at least one failed build caused by flaky tests [10].

Previous studies have identified and categorized potential causes of flaky tests across different programming languages, including Java [11], Python [12], and JavaScript [13]. Luo et al. [11] reported the first extensive empirical study on flaky tests in open-source software, and they categorized the causes and fixing strategies for these flaky tests. They found that one of the major causes of flaky tests is test order dependency—that is, the tests' outcomes depend on the order in which they are executed. Flaky tests that pass in one order but fail in another are known as *order-dependent tests*. While test order dependency is acknowledged as one of the top causes of test flakiness in several languages, including Java and Python [6], [11], [12], [14], the number of order-dependent tests reported in JavaScript projects is low [13]. However, these findings were based on the reported flaky tests in open-source projects (i.e., extracted from issue trackers). The projects may contain order-dependent tests that the developers have yet to discover, potentially failing and misleading developers later during development. Indeed, the most popular JavaScript testing framework, Jest [15], [16], introduced a feature that allows developers to randomize the order in which they run Jest tests due to concerns from developers about the potential of having order-dependent tests [17]. This feature is still relatively new (made available in late 2023) and not widely adopted, meaning many projects may still contain order-dependent tests that developers are unaware of.

In this study, we conduct an in-depth investigation into order-dependent tests in JavaScript, focusing on the Jest testing framework. Jest offers many features, such as parallelization and ordered test execution, but tests may still share state between each other, leading to order-dependent tests. Our goal is to understand the prevalence of order-dependent tests within popular open-source JavaScript projects after actively searching for them. We also want to understand the reasons for their failures, i.e., what state these tests share between them. The results of our study can provide better insights towards how developers can manage order-dependent tests in their test suites, debugging their root causes to eventually fix them so they no longer have different test outcomes in different orders.

The main contributions of this paper are the answers to the following two research questions:

- **RQ1** How can order-dependent tests manifest their failures in JavaScript projects?
- **RQ2** What are the main causes of order-dependent tests?

Accepted for publication by IEEE. © 2025 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/ republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

RQ1 aims to investigate ways that order-dependent tests' failures can manifest in Jest. We employed a multi-layer reorder and rerun approach to uncover order-dependent tests in Jest. By altering the sequence in which the tests are executed within programs/test suites and rerunning them multiple times, we observe whether the test outcomes change. RQ2 aims to understand the main causes of order-dependent tests in Jest. We manually inspected each detected order-dependent test and classified the cause of flakiness into different categories.

To answer our research questions, we developed a tool called `JS-TOD`, which extracts, reorders, and reruns tests and test suites using user-defined numbers of reorders and reruns. We evaluated on a dataset of 81 projects. Using `JS-TOD`, we reordered tests and test suites 10 times each and rerun them 10 times each. Our results show that order-dependent tests are possible despite Jest executing tests sequentially (which is supposed to guarantee the default test order).

Of the 55 detected order-dependent tests, 52 were found between individual tests, and three were detected between describe blocks within the same test suite rather than between different test suites. We found that the main causes of those flaky tests are shared files (e.g., access to a shared file created in one of the tests and accessed by other tests) and shared mocking state (e.g., some mocked state was changed by tests).

II. Background

A. Test Order-Dependency

Ideally, tests should function independently of each other and produce the same results regardless of their order of execution [18]. In reality, tests often have different outcomes when executed in different orders [5], [18]. These tests are called *order-dependent tests*, and previous studies found them to form a significant proportion of flaky tests [11], [12].

Prior work classified order-dependent tests into one of two categories based on their result when run in isolation: victim or brittle [19]. A *victim* passes when run in isolation but fails when run after another test(s), known as a *polluter*. Essentially, a polluter "pollutes" the state shared between the two tests, leading the victim to run in an improperly initialized state, resulting in a failure. On the other hand, a *brittle* test fails when run in isolation but passes when run after another test(s), called a *state-setter*. Essentially, the state-setter is setting up the initial state the brittle needs to start in for it to pass; otherwise, the brittle fails as it cannot run properly on its own.

Listing 1 shows an example of an order-dependent test from "react-testing-library" [20]. Test "first" is a state-setter for test "second," which is a brittle test. Running the test "second" before the test "first" or in isolation will result in a test failure as the document does not have the proper body to be checked.

```
1 test('first', () => {
2   render(<div>hi</div>)})
3 test('second', () => {
4  expect(document.body.innerHTML).toEqual('<div↩
       ><div>hi</div></div>')})
```

Listing 1. Order-dependent tests from "react-testing-library"

B. Jest

Jest is the most popular JavaScript testing framework [16], [21]. In Jest, a *test suite* corresponds to a JavaScript file that contains the tests to be run, so there can be many different test suites in a project. A test suite may encompass multiple testing blocks. A developer may choose to use `describe` blocks to group together multiple related tests [22], introducing a level of organization for their tests. The `test` and `it` functions define individual tests within the test suites, each tasked with verifying a specific scenario. The `describe` block allows for hierarchical organization, while the `test` function outlines the specifics of each case, contributing to a formalized and systematic testing approach. Note that individual tests do not always need to be contained with a `describe` block, but if there are `describe` blocks, then tests within the same `describe` block are executed with each other without interleaving with tests in other `describe` blocks.

Jest has different approaches to running tests and test suites. By default, Jest runs test suites in parallel rather than sequentially. By doing so, Jest does not guarantee the running order of test suites [23]. Jest assigns a worker to run each test suite. A developer can use the option `maxWorkers` to limit the number of workers or use the `runInBand` option to have the test suites run sequentially. Jest determines, based on previous runs, if it will be faster to run test suites sequentially rather than in parallel without informing the developer [24]. It also orders the test suites to run based on past runtime, failure history, and/or file size (if no prior information is available) [15]. Since these factors can change at any time, Jest may arbitrarily determine to run test suites sequentially and in some non-deterministic order, potentially leading to order-dependent tests that fail when Jest happens to run them in a failing order.

On the other hand, Jest runs tests inside test suites in the order they are encountered in the file (i.e., the first test in the file runs first). If there are `describe` blocks to group tests, Jest runs them in the order they are defined within the file [25]. Jest runs all the `describe` blocks in a test suite before executing any actual tests [25].

Jest has different options for test parallelization and selection [26]. For example, the `testNamePattern` option allows running only individual tests or `describe` blocks that match a specific naming pattern. The `shard` option allows splitting a test suite into multiple shards and then running specific shards, which is useful for parallelizing test execution across multiple machines or environments to reduce the overall test runtime (each shard is run on different machines). The `randomize` option was released in late 2023 in response to a popular feature request from developers [17], who indicated the need for test randomization that is common in other testing frameworks such as JUnit and NUnit. From the discussion of this feature, we observed that developers saw that randomizing test execution helps uncover cases where tests inadvertently depend on the state left by previous ones.

In Jest, test dependencies can manifest at various levels. Order dependency may arise between tests, `describe`

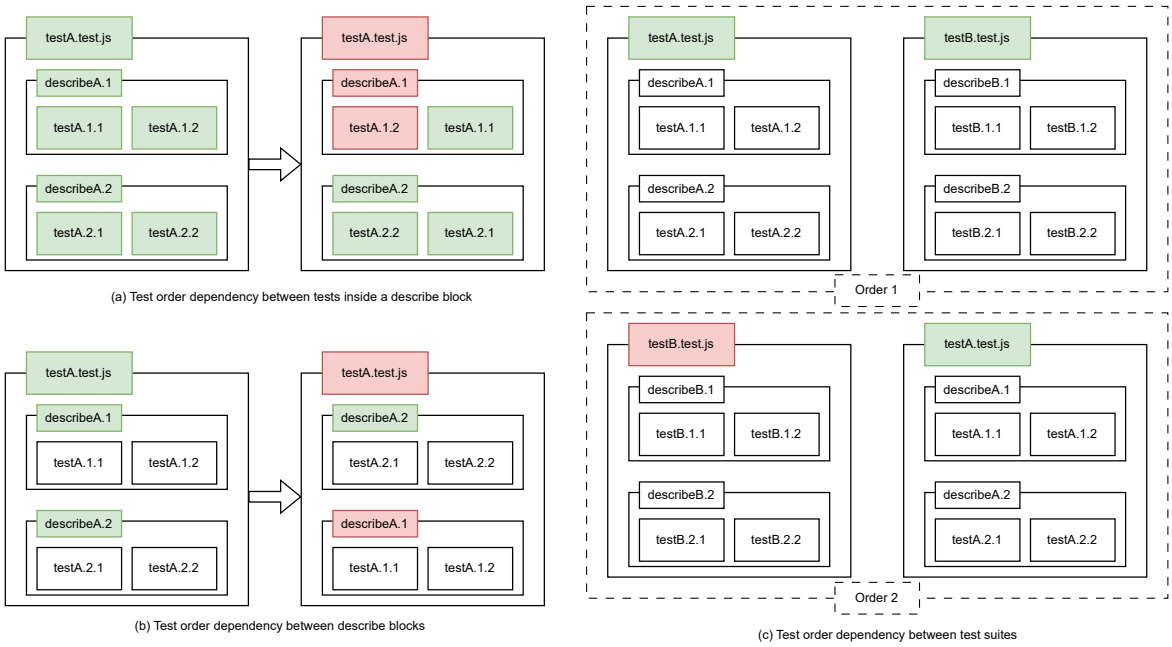


Fig. 1. Different levels of test order dependency with passed tests in green and failed tests in red

blocks, and test suites. Fig. 1(a) illustrates an example of order dependency between two tests. Changing the order of `testA.1.1` and `testA.1.2` results in the failure of `testA.1.2`, as it depends on `testA.1.1`. The order dependency between `describe` blocks is demonstrated in Fig. 1(b), where running `describeA.2` before `describeA.1` leads to the failure of `describeA.1`. Fig. 1(c) depicts order dependency between two test suites: `testA.test.js` and `testB.test.js`. The passing order for these two tests is `testA.test.js` → `testB.test.js`. By changing the running order to `testB.test.js` → `testA.test.js`, `testB.test.js` will fail.

III. Approach

We implement JavaScript Test Order-dependency Detector (`JS-TOD`) to detect order-dependent tests within JavaScript projects using Jest. `JS-TOD` first extracts all test paths from a given project and then reorders the tests and test suites using a user-defined number of reorders. `JS-TOD` runs all newly reordered tests for the given number of re-runs and finally exports the results (i.e., test outcomes) into JSON files.

To reorder tests at all three levels (Fig. 1), we implement a randomization test reordering approach, shown in Algorithm 1. The function `randomizer` takes as input an array, representing the set of test suite paths, tests, or `describe` blocks to be reordered, and an integer, representing the number of reorders. The function returns the set of reorders. If all possible permutations of items are smaller than the provided number of reorders, the function returns all permutations. Otherwise, it shuffles the items to create a new reorder and adds it to the set of reorders to be returned if it is unique. This process repeats until reaching the input number of reorders.

Algorithm 1 Tests reordering approach

```
1:  function RANDOMIZER(array, reorder_num)
2:      all ← calcNumPermutations(array_length)
3:      if reorder_num ≥ all then
4:          result ← getPermutations(array)
5:      else
6:          while result_length ≤ reorder_num do
7:              newArray ← shuffleArray(array)
8:              if isUnique(result, newArray) then
9:                  result ← newArray
10:             end if
11:         end while
12:     end if
13:     return result
14: end function
```

`JS-TOD` uses different approaches to ordering test suites, tests and `describe` blocks:

A. Reordering Test Suites

For reordering test suites (Fig. 1(c)), we need to extract the paths of test suites from each project and pass the running order to Jest. We extract the test suites' paths for a given project automatically using Jest's `-listTests` option [26].

Then, we pass the test suite paths to the `randomizer` function in Algorithm 1 to generate up to `reorder_num` unique reorders. In addition, Jest uses a `TestSequencer` [27] class to determine the order in which to run the test suites. We write a custom sequencer that extends this `TestSequencer` class, allowing us to run the test suites in a specific order. Listing 2 shows our custom sequencer, which

receives the running order as an argument and returns tests in the defined order. We provide the custom sequencer and running order to Jest by passing the path to the custom sequencer class after the `-testSequencer` option and order of test suites after the `-order` option, e.g., `npx jest --runInBand --testSequencer='/path/to/customSequencer' --order='/path/to/test1,/path/to/test2'`.

```
class CustomSequencer extends TestSequencer {
 sort(tests) { //get the running order and put it in orderPath array
  const orderPathString = process.argv.find ((arg)=>arg.startsWith("--order")).replace("--order=","");
  const orderPath =orderPathString.split(",");

  return tests.sort((testA, testB) => {
   const indexA =orderPath.indexOf(testA.path);
   const indexB =orderPath.indexOf(testB.path);
   if (indexA === -1 && indexB === -1) return 0; // Keep both tests in their original order if not specified in orderPath
   if (indexA === -1) return 1;
   if (indexB === -1) return -1;
   const result = indexA - indexB;
   return result
   ;})
 ;}}
```

Listing 2. CustomSequencer class

B. Reordering Tests and `describe` blocks

Algorithm 2 describes the process for extracting and reordering both tests and `describe` blocks (Fig. 1(a),(b)). After extracting test suites, we use the Babel toolchain [28], a JavaScript compiler and source code transformer, to parse each test suite. Babel generates an Abstract Syntax Tree (AST) for each test suite (including nodes, parameters and types). Using the AST, we identify all `describe` blocks within each test suite of a given project. The `describe` blocks are recognised by recording AST nodes when a node's type is "*identifier*" and its name is "*describe*". These blocks are then reordered and saved in new test files in the original directory. For identifying tests inside each `describe` block, we use the same randomizing approach as for Algorithm 2, except the identifier names of these AST nodes are either `it` or `test`.

We record the results of each run in a JSON file for further investigation. In cases where a test suite consistently fails across all reruns of the same order, we categorize it as an order-dependent test. Otherwise, if it passes and fails even as the order remains the same, the test suite is flaky, depending on other non-deterministic factors.

IV. Experimental Setup

Fig. 2 shows an overview of our experiment, including data collection and analysis processes. For our evaluation, we reorder each test suite, test, and `describe` block for 10 reorders and reran each new test suite 10 times using `JS-TOD`.

A. Dataset

To collect a dataset of JavaScript projects for evaluation, we utilise GitHub's REST API [29] to search for projects.

Algorithm 2 Extracting and reordering `describe` blocks/tests in `JS-TOD`

Input: a project path
Output: JSON file saved in the project path

$INSTALL_PROJECT(project_path)$
$testSuites \leftarrow ListTestSuites(project_path)$
for `testSuite in testSuites` **do**
  $blocks \leftarrow PARSE(testSuite)$
  if $len(blocks) \geq 2$ **then**
    $newTests \leftarrow randomizer(blocks, reorder_num)$
    for `newTest in newTests` **do**
      $newTestCode \leftarrow generateCode(newTest)$
      $name \leftarrow generateName(newTestCode)$
      $save(name, newTestCode)$
    end for
  end if
end for
for `newTestSuite in newTestSuites` **do**
  $RUN(newTestSuite, rerun_num)$
end for

function PARSE(testSuite)
  $ast \leftarrow babel_parser(testSuite)$
  $blocks \leftarrow TRAVERSE(ast, type)$
  return $blocks$
end function

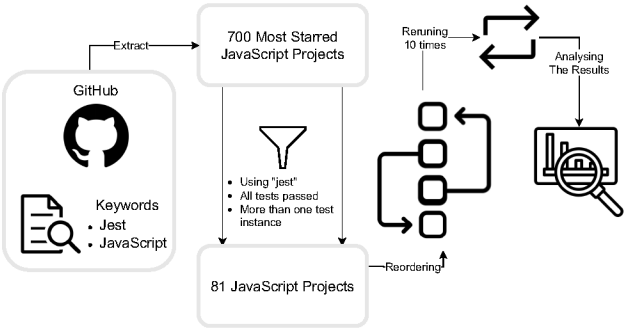


Fig. 2. An overview of the data collection and analysis process

We use a keyword search to collect projects that contain "jest" in the repository (e.g., topics, issues, and README files). In addition, we searched for JavaScript projects without any special keywords. In both search queries, we order the projects based on GitHub's stars, where there is a correlation between the number of stars and the number of forks and contributors, demonstrating the project's popularity [30]. Based on our search results, we select the 700 most starred projects to form a large enough initial dataset for inspection. After cloning all 700 projects, we select the projects that use Jest as their testing framework by searching with the keyword `jest` in the `package.json` file. We find 304 projects that use Jest.

For each project, we execute all tests in the default order (i.e.,

the original test order as they appear in the project). After each execution, we record the number of test suites, `describe` blocks, and tests for further investigation. We then exclude the projects that do not have enough test instances to investigate (a project should contain at least two tests so that we can reorder them). We find that 82 of the 304 selected projects have more than one test instance (at least two tests, `describe` blocks, or test suites) and all passing tests when run in the default order. We exclude all other projects from our evaluation.

When we run `JS-TOD`, we find a project ("snapshot-diff") where the reordering does not work. The project has a test with a missing semicolon that causes Babel to fail in transforming the tests. Hence, we exclude this project from our evaluation and continue with the remaining 81 projects. Table I shows summary statistics of the selected projects.

TABLE I
PROJECT STATISTICS

	Production		Tests			
	LOC	#files	LOC	#tests	#describe blocks	#test suites
Mean	1791	32	690	79	18	11
Total	145092	2583	55911	6398	1420	873

B. Environment

We conduct our evaluation on a machine with a 2.3 GHz Intel Core i9 processor with 16 GB RAM running on a macOS 14. We use `npm` version 9.5.1 and `Node.js` version 18.16.1. The experiment took approximately 121 hours to run across all projects and reruns required (see Table II for runtime data).

C. Manual analysis

We conduct two different manual analyses on the collected results. We conduct our first manual analysis on the tests before reordering anything. The goal is to check if all tests of a project pass by default, that all test paths are available and correctly extracted by Jest's `-listtests` option (available from Jest version 20.0.0 [31]). In eight projects, the *-listTests* option could not list the test paths due to using older versions of Jest. In those projects, we manually extract the test paths to pass to `JS-TOD`, allowing us to evaluate on those projects.

After reordering the tests, we conduct a second manual analysis to verify the cause of failures for failing tests. One of the authors manually inspected all results, examining the source code and error messages to identify instances of failed orders. After identifying the cause of failure, a second author independently inspected the code to confirm if it was due to an order-dependent test. The authors discussed cases where there were disagreements, and in one case, a third author confirmed the cause of flakiness. Our goal is to create a categorization of common reasons for flakiness, namely the types of shared state between tests, among the detected order-dependent tests.

We make publicly available our `JS-TOD` implementation and a full replication package for our evaluation, which includes all data, scripts and full results [32].

V. RESULTS

We apply `JS-TOD` on the tests within each of the 81 projects we evaluate, reordering tests at the three levels of tests, `describe` blocks, and test suites.

A. RQ1. Manifesting order-dependent tests

Table II shows the results of applying `JS-TOD` on each project. The total number of projects in Table II is the summation of all projects in the three different levels that overlap with each other in some projects.

In total, we find 55 order-dependent tests across 10 projects. All 55 tests fail across all 10 reruns of the same order, confirming that they indeed are order-dependent tests. The majority of these order-dependent tests were found by reordering at the level of tests (52 tests). Only three order-dependent tests were found at the level of `describe` blocks.

1) Order dependency between tests: From all 81 projects, we find that 12 of those projects contain only a single test inside each `describe` block, and thus, we do not include them in our analysis of tests. Among others, running `JS-TOD` on project "jest-serializer-vue" was too time-consuming (more than six hours), and "draggable" has a test with a missing semicolon that causes Babel to fail in transforming the test code. Therefore, we exclude these 14 projects and continued with the remaining 67 projects for the test-level analysis.

JS-TOD successfully reorders the tests for 85% of all projects (57 out of 67 projects), with the remaining projects fixed manually. Our analysis reveals a total of 52 order-dependent tests from nine projects. As shown in Table III, most of the flaky tests appear in project "jest-webextension-mock" with 35 order-dependent tests, followed by "serverless-jest-plugin", which has five order-dependent tests, and "native-testing-library" with four order-dependent tests. The rest of the six projects contain one or two order-dependent tests.

2) Order dependency between `describe` blocks: We notice that 47 out of the 81 projects have no or just a single `describe` block inside each test suite, and therefore, it is not possible to reorder the `describe` blocks for such projects. In addition, "draggable" has a test with a missing semicolon that causes Babel to fail in transforming the project's tests. As a result, we continue with the 33 projects for further analysis.

In some cases, the `describe` blocks can be nested (i.e., a `describe` block can contain other `describe` blocks within its structure). For the 33 remaining projects, we reorder the `describe` blocks up to two nested levels. One project, "Jasmine-Matchers" [33] has more than two nested blocks, and we could not find a suitable way to reorder its blocks given technical challenges in rewriting those tests, so we exclude this project from our analysis. Ultimately, we analyse 32 projects.

Among the 32 projects, we find one project, "jest-build-ins", with two order-dependent `describe` blocks (Table IV), namely "native jest mock function inspectors" and "exposing mock internals".

3) Order dependency between test suites: We only analyse projects with multiple test suites to determine order dependency between test suites, resulting in 62 out of 81 projects. In

TABLE II
RESULTS OF REORDERING AND RERUNNING TESTS IN THREE LEVELS

Reordering Level	#projects	Default Order		Reordering Result						
		#tests	#test suites	#test	#test suites	#failed tests	#failed test suites	#ODFT*	#ODFTS**	runtime (s)***
Test	67	6169	782	44713	4353	108	65	**52**	**19**	273788.1
Describe block	32	5818	692	18534	558	11	6	**3**	**1**	116013.6
Test suite	49	5960	850	59600	8500	0	0	0	0	44453.3
Total	148	17947	2324	122847	13412	119	71	**55**	**20**	434255.0

* order-dependent flaky tests
** order-dependent flaky test suites
*** runtime calculated for the total of 10 reorders each executed 10 times (100 runs in total)

13 of the 62 projects, we encountered an issue where the `-testSequencer` was unrecognized in the `npx` command due to using older versions of Jest. Since we rely on Jest's test sequencer class to enforce a specific running order, we cannot reorder the tests for these projects. All test suites pass for the remaining 49 projects regardless of their test-suite order. Thus, we do not find any order dependency between test suites.

B. RQ2. Causes of order-dependent tests

We present the results of our root-cause analysis on the detected order-dependent tests at the test level and at the `describe` block level.

1) Order dependency between tests: Table III shows the results of our analysis on detected order-dependent tests at the test level. In total, 56 projects have all tests passing across all 10 reorders for 10 reruns, while 11 projects experienced at least one failed test after being reordered (in one of the 10 reruns). We manually investigate the results of those 11 projects to identify the cause of test failures. We identify 52 order-dependent tests from the nine projects that have them (shaded rows in Table III).

We find the causes for the order-dependent tests can be divided into two categories: sharing file and mocking state. Among the detected order-dependent tests, we identify 13 tests whose failures are caused by file sharing across seven projects, and 39 tests caused by shared mocking state between tests across two projects.

File sharing: We find that developers use different methods to share a file between tests. For example, we identify one order-dependent test within the "buildJsonResults" test suite in "jest-junit", which is related to two tests: "should report no results as error" and "should honour templates when the test has errors". The two tests shared a mock file (as seen in Listing 3 - lines 2 and 7). The first test sets `reportTestSuiteErrors: "true"` on line 4, which alters the cached object of the mock file used by `require` (lines 2 and 7) and consequently impacts the outcome of the subsequent test. On line 11, the test checks whether the total number of errors is equal to one. In case the second test runs first, the `totals.errors` becomes not equal to one, therefore the test fails. Consequently, we notice that whenever "should honour templates when the test has errors" executes before "should report no results as error", the latter test fails. Hence, "should honour templates when the test has errors" is a polluter and "should report no results as error" is a victim.

```
1  it('should honor templates when test has ↩
       errors', () => {
2     const failingTestsReport = require('../↩
          __mocks__/failing-compilation.json');
3     ...
4     reportTestSuiteErrors: "true",
5     ...
6  it('should report no results as error',() => {
7     const failingTestsReport = require('../↩
          __mocks__/failing-compilation.json');
8     ...
9     reportTestSuiteErrors: "true"
10    ...
11    expect(totals.errors).toEqual(1);
```

Listing 3. Tests that shared a file from jest-junit

Another example of file sharing is in "jest-image-snapshot". Some of the tests create/update image snapshot files accessed by other tests. Specifically, the test "should write the image snapshot on first run" writes files to the shared location. The tests "should not delete the snapshot when environment flag is not enabled" and "should delete the snapshot when environment flag is enabled" expect those files to exist. That is, the latter two tests depend on the state set up by the first.

```
1  function getSnapshotFiles() {
2   return fs.readdirSync(path.join(tmpDir, '↩
        __image_snapshots__'));}
3  it('should write the image snapshot on first ↩
       run',()=>{
4   const{status, stdout, stderr}=runJest(['-u']);
5   expect(stderr).toContain('snapshots written');
6   expect(status).toEqual(0);
7   expect(stdout).toEqual('');
8   expect(getSnapshotFiles()).toHaveLength(3);});
9  it('should not delete the snapshot when ↩
       environment flag is not enabled', () => {
10  const { status, stdout } = runJest(['-u', '↩
        image.test.js']);
11  expect(status).toEqual(0);
12  expect(stdout).toEqual('');
13  expect(getSnapshotFiles()).toHaveLength(3);});
14 it('should delete the snapshot when ↩
       environment flag is enabled', () => {
15  const { status, stdout, stderr } = runJest(['-↩
        u', 'image.test.js'], {
16   JEST_IMAGE_SNAPSHOT_TRACK_OBSOLETE: '1',});
17  expect(stderr).toContain('outdated snapshot');
```

TABLE III
RESULTS OF REORDERING TESTS IN 67 PROJECTS

Project	Default Order		Reordered tests							
	#tests	#tests suites	#test	#test suites	%failed test	%failed tests suites	#ODFT	#ODFTS	OD cause	runtime(s)
jest-image-snapshot	100	5	980	46	5.0%	35%	**2**	**1**	file sharing	12404.5
jest-junit	53	6	496	34	7.0%	18%	**1**	**1**	file sharing	1201.6
jest-when[#]	86	1	860	10	1.0%	100%	0	0	-	820.1
jest-webextension-mock	173	14	391	96	38.0%	68%	**35**	**7**	shared mocking state	7491.8
commander.js[#]	1108	102	10189	867	0.1%	1%	0	0	-	54987.3
jest-mock-now	5	1	46	6	8.7%	67%	**1**	**1**	file sharing	411.8
KaTeX	1216	8	8020	62	0.2%	16%	**2**	**1**	file sharing	8174.6
native-testing-library	159	36	949	196	2.0%	4%	**4**	**1**	shared mocking state	14322.9
react-testing-library	237	14	392	75	0.3%	1%	**1**	**1**	file sharing	2329.3
serverless-jest-plugin	30	6	288	52	10.0%	54%	**5**	**5**	file sharing	6507.7
vue-testing-with-jest-conf17	11	3	78	14	6.0%	36%	**1**	**1**	file sharing	544.6
Total	3178	196	22689	1458			52	19		109196.2

[#] shaded rows are projects with failing tests but not order-dependent flaky tests

```
18  expect(status).toEqual(0);
19  expect(stdout).toEqual('');
20  expect(getSnapshotFiles()).toHaveLength(1);});
```

Listing 4. Tests with shared files between them from jest-image-snapshot

Shared mocking state: Mocking states is a type of state sharing that specifically looks at the state of a mock. One example of sharing mocking states is "jest-webextension-mock". The project has seven test suites with 35 order-dependent tests. All 35 tests fail after reordering those tests due to the issue of checking the number of times functions are called.

An example from the project is shown in Listing 5. The "command" test suite includes two tests: "getAll" and "getAll promise". In "getAll", when the function `browser.commands.getAll()` is called (line 3), the `expect` statement anticipates the function to be called once (line 4). The "getAll promise" test calls this function (line 7). If "getAll promise" runs before "getAll", the function gets called twice, causing the `expect` statement in line 4 to return false and a test to fail. A possible fix is to use `jest.clearAllMocks()` to clear the mocking state in a `beforeEach` hook, which ensures that the mocks are cleared and the mock state is not shared between tests.

```
1  test('getAll', (done) => {
2   ...
3    browser.commands.getAll(callback);
4    expect(browser.commands.getAll).↩
         toHaveBeenCalledTimes(1);
5   ...
6  test('getAll promise', () => {
7   return expect(browser.commands.getAll()).↩
         resolves.toBeUndefined();
```

Listing 5. Tests with shared mocking state from jest-webextension-mock

Listing 6 shows another example of order-dependent tests caused by a shared mocking state. The test suite has five tests, of which four are expected to use the mock function on line 2. The remaining test, "async act should not show an error when ReactTestUtils.act returns something", uses the same mock function with a new and different implementation specific to the test (line 12). In the default order, test "async act should not show an error when ReactTestUtils.act returns something" is the last test in the test suite, meaning it will run last, and therefore it does not share its inner mocking function with other tests. As a result of reordering tests, the `jest.mock('react-test-renderer')` defined in this test will impact the other four tests as the mocking state changes, which causes the other tests that come after the target test to fail.

```
1  let asyncAct;
2  jest.mock('react-test-renderer', () => ({
3    act: cb => { ...}}))
4  beforeEach(() => {
5    jest.resetModules();
6    asyncAct = require('../act-compat').asyncAct;
7    jest.spyOn(console,'error').↩
         mockImplementation(()=>{});})
8  afterEach(() => {
9    console.error.mockRestore();})
10 test('async act should not show an error when ↩
       ReactTestUtils.act returns something', ↩
       async () => {
11   jest.resetModules();
12   jest.mock('react-test-renderer',()=>({...
```

Listing 6. Tests with shared mocking state from native-testing-library

Other failed tests: We observe two projects with failing tests after the reordering process: "jest-when" [34] and "commander.js" [35], but we do not find the failing tests to be order-dependent tests. In "jest-when," we observe test logic that depends on the test name and file structure. For example, as shown in Listing 7, test "fails verification check if all mocks were not called with line numbers" (line 5) depends on the name of the test suite. The tests fail when we transform them into new suites (i.e., the newly reordered test files). Therefore, we attribute the failure to the structure of the test code and not test order dependency. In addition, "commander.js" experiences failed tests when rerun in some orders. After analysis, we discovered these failures are false positives, more related to resource issues than test order dependency.

```
    try {
    verifyAllWhenMocksCalled();
  } catch (e) {
    const errorLines = e.message.split('\n');
    const currentFilePathPattern = /src↩
        (?:\\|\/)when\.test\.js:\d{3}(.|\s)*/;
```

Listing 7. Tests from jest-when

There were also nine projects with failing test suites after reordering the tests. After manually analyzing the results, we find that the failures occur due to Babel not parsing some parts of the original test suites, and thus, JS-TOD ends up not adding them to the new test suites. For example, Listing 8 from "jest-it-up" [36] shows part of a test suite that has a `require` statement in line 2. JS-TOD does not identify this statement when traversing the AST of the original test suite and does not include it in new test suites after reordering. This parsing error causes the test "returns the new contents, and changes if new thresholds" to fail. As a result, we manually fix the code of the new test suites by adding the missing code segments and then comparing those with the original suites to ensure that it is identical. After this manual modification, we rerun all test suites, resulting in all tests passing.

In addition, there are two projects ("jest-fail-on-console" and "KaTeX") with specific Jest configurations for running tests with defined name patterns. As we add reordered tests to the new test suite, we change Jest's configuration to run them.

```
1 const fs = require('fs')
2 require('ansi-colors').enabled = false
3     ...
4 it('returns the new contents and changes if ↩
      new thresholds', () => {...
```

Listing 8. Example of unidentifiable code by AST from jest-it-up

2) Order dependency between `describe` blocks: Table IV shows the projects with failing tests after reordering `describe` blocks. The results show that all reordered `describe` blocks passed for all projects except three: "jest-when", "commander.js" and "fetch-mock-jest". As mentioned in Section V-B1, "jest-when" has a test that relies on the names and structures of the test suites. Failing tests in "commander.js" are caused by resource-related issues. Hence, we do not attribute the failed tests to order-dependent flakiness.

"fetch-mock-jest" [37] has two order-dependent `describe` blocks. The `describe` blocks contain three tests that expect a function to be called a set number of times (lines 5, 9 and 13 in Listing 9). As a result of this shared mocking state, changing the orders of `describe` blocks leads to tests failing.

```
1  describe('exposing mock internals', () =>{
2      ...
3   it('exposes `calls` property', () =>{
4    expect(fetch.mock.calls).toBeDefined();
5    expect(fetch.mock.calls.length).toBe(2);
6    ...
7   it('exposes `results` property', async () =>{
8    expect(fetch.mock.results).toBeDefined();
9    expect(fetch.mock.results.length).toEqual(2);
10   ...
11 describe('native jest mock function inspectors↩
       ', () => {
12  it('.toHaveBeenCalled()', () => {
13      expect(() => expect(fetch).toHaveBeenCalled↩
            ()).not.toThrow();
```

Listing 9. `describe` blocks from fetch-mock-jest

VI. Discussion

A. Order dependency flakiness in JavaScript:

Our investigation of order-dependent flakiness in JavaScript reveals only 55 order-dependent tests from 10 projects. We observe a notable low number of order-dependent tests in JavaScript projects compared to other studied languages, particularly Python [12], [38] and Java [11], [39], where order-dependent tests are among the most prevalent types of flaky tests. Our results confirm findings from an empirical study of flakiness in JavaScript projects from issue tracking data, which also found very few cases of order-dependent tests in JavaScript projects [13] (across multiple testing frameworks, including Jest). We attribute this low number of order-dependent tests to two main factors: 1) the way that JavaScript developers organize their tests when using Jest (that is, relevant tests are grouped together within the same blocks within the same test file), and 2) the way that testing frameworks like Jest handle test order (running test suites in parallel by default or the test and `describe` blocks in the order they appear in the file).

There are no major factors in JavaScript that distinguish it from other languages in terms of how it can cause test order dependency. In Java, order-dependent test flakiness caused by state-sharing is mostly due to modifying shared static fields [40]. Since the release of version ES6, JavaScript has provided support for static fields in classes and modules that can export properties. Prior to ES6, developers used the prototype property to emulate them. Our study does not reveal any order-dependent tests due to state sharing through these mechanisms. This result could be due to how programmers write JavaScript code (i.e., in a more functional style) or the use of these features in programs. Future work can investigate the types of state sharing in relation to language features.

Test runners ensure the ordered execution of tests in test classes/suites. For Java projects, the JUnit testing framework can execute tests concurrently in the same JVM process. Jest, on the other hand, runs tests in separate processes rather than in the same process, due to JavaScript being single-threaded by nature. Jest uses node worker threads or child processes to parallelise test executions [41].

B. Implications

This study has implications for both the JavaScript developers/testers (who write test code) and designers of test management tools. The results show that order-dependent tests manifest largely between tests rather than test suites or `describe` blocks. While Jest by default ensures tests in the same test suite run in the declared order, developers are still keen to ensure test independence and even requested Jest to introduce a randomization feature to stress their testing [17]. Our implementation in JS-TOD not only randomizes the order of tests in a test suite but also between `describe` blocks and

TABLE IV
RESULT OF REORDERING `describe` BLOCKS

Project	Default Order		Reordered tests							
	#tests	#tests suites	#test	#test suites	%failed test	%failed tests suites	#ODFT	#ODFTS	OD cause	runtime(s)
jest-when	86	1	172	2	1.0%	100.0%	0	0	-	417.4
commander.js	1108	102	2590	122	0.2%	2.9%	0	0	-	14616.0
fetch-mock-jest[#]	216	9	2076	76	0.4%	5.3%	**3**	**1**	shared mocking state	4320.0
Total	1410	112	4838	200			3	1		19353.4

[#] shaded rows are projects with order-dependent flaky tests

test suites, giving greater exploration options for developers to use. Indeed, we even detected some dependencies between `describe` blocks in our evaluation.

Similarly to previous studies done on Java and Python [11], [12], [18], [42], [43], we observe that order-dependent tests share states via external resources such as files. Testers should be vigilant about reusing files between tests. However, we find another major reason for order-dependent test flakiness is the sharing of mocking state, which was not discussed as a major reason in past studies for other languages. In Jest, modules mocked using `jest.mock` function are intended to be mocked within the test suite that calls the mocking function [44], so their state persists between tests unless reset with `jest.resetModules()` or manually unmocked. Although `jest.resetModules()` resets the module registry, it does not automatically reset all mocks that were applied globally. `jest.resetModules()` only resets the module cache, meaning any new `require()` calls will reload the module. However, if a module is mocked globally using `jest.mock()`, it still returns the mocked version, because the mock was applied outside the scope of the individual test.

We have conducted several experiments to explore ways to fix and mitigate flaky order-dependent tests, especially those caused by the shared mocking state. We found that one possible strategy for fixing shared mocking states is to ensure that each test starts with a fresh and clean mock state. To achieve this, one must explicitly reset the mock functions before or after each test. In the example provided in Listing 6, resetting `react-test-renderer` in the `beforeEach` block can help avoid the flakiness as it cleans the mock before each test. Additionally, calling the `jest.clearAllMocks()` function before each test can effectively clear mocks before execution. By clearing all mocking states in a `beforeEach` block, we successfully fixed 34 out of the 39 order-dependent tests that are caused by shared mocking states.

Similar to most other testing frameworks, Jest also provides test fixture features (setup and teardown functions that get called before and after tests, respectively) that developers should use to eliminate order-dependent test issues, ensuring file and mock states are reset between tests.

VII. THREATS TO VALIDITY

Number of runs and orders: Deciding on how many reruns are required to reveal if a test is flaky has been pointed out as a significant issue in test flakiness research [45]. We decided to use 10 reorderings and 10 reruns for each new order (i.e., 100 reruns for each test suite). Based on our pilot runs and our previous experience from running flaky tests, we noted that with 10 reruns, we can observe the same results as increasing the number of reruns to 20 or 30. While increasing the number of reorders and reruns can potentially lead to finding more order-dependent tests, our experimentation with increased reorders and reruns showed no changes to the results.

Reproducibility of results: Our results from each program may not be easily reproducible as we used a randomization approach to reorder tests (i.e., each time we reorder the tests, we may get a different order). However, it is possible to reproduce the same results by running the same order we found to be flaky (see our replication package for more details [32]).

Generalizability of results: The results are based on analyzing JavaScript programs that use Jest as their testing framework. We cannot guarantee that the results will hold for JavaScript programs that use other testing frameworks. Jest is the most popular framework and thus is considered representative. While there are similarities between the testing frameworks, one must adjust the approach using other testing frameworks, which requires updating the tool to the framework's options and settings. For example, not all testing frameworks have the concept of `describe` blocks, so they would not have order-dependency between these blocks as we observed in Jest.

Manual analysis: Part of the project selection and the categorization of causes was done manually, which may affect construct validity due to oversight and bias. Two authors independently examined the source code and error messages to reduce the likelihood of false positives when categorizing the cause of flakiness. Whenever there was disagreement, we implemented a conflict resolution step where a third author was then involved to independently classify the disagreement. For all cases, all three authors rigorously discussed the causes until 100% agreement was reached.

VIII. RELATED WORK

Previous work has investigated various aspects of flaky tests, such as root causes and ways to identify, detect, predict, or mitigate flaky tests [6], [14], [45]. Several previous studies on test flakiness have largely focused on specific causes, such as test order dependency [46], async wait and timing dependency [47], [48], concurrency [49], unordered collections [50], random

numbers [51], [52], code instrumentation [53] or network flakiness [54]. Of those cases, test order dependency has been emphasized as a major cause of test flakiness. The earlier work of Luo et al. [11] found that test order dependency is a major cause of flakiness in the analyzed Apache Java projects. Similarly, Gruber et al. [12] reported that test order dependency is the main cause of flakiness in Python programs.

A. Order dependent test flakiness

Several studies have investigated test order dependency in different contexts [19], [38], [39], [43], [55]–[58]. Lam et al. [39] developed a framework, iDFlakies, which detects and classifies Java tests into order-dependent and non-order-dependent by running them in randomized orders. They found that 50.5% of flaky tests in their study were caused by test order deficiency. IncIDFlakies [58] improved upon iDFlakies by analysing code changes to detect newly introduced order-dependent flaky tests. The tool identifies and selects only tests with shared states between them, as such tests can potentially result in flakiness due to test-order dependency.

Shi et al. proposed iFixFlakies [19] to automatically fix order-dependent tests in Java. It relies on iDFlakies [39] to detect order-dependent tests. Then, it reruns them in certain orders to determine whether an order-dependent test is a victim, polluter or brittle. iFixFlakies searches for cleaners, which are tests that clean polluted state between tests, and uses their code to generate patches for repairing order-dependent tests. Wang et al. used the same approach in iPFlakies [38] for detecting and fixing order-dependent tests in Python. Li et al. [57] later proposed ODRepair, an automated technique to repair order-dependent tests by determining the polluted shared state that results in the order-dependent test failing. It then generates code that can modify and reset the shared state so that the order-dependent test can pass.

Gruber et al. [43] developed FlaPy, a tool to repeatedly execute the test suites of a given set of Python projects. FlaPy can categorize tests into four groups: not flaky, non-order-dependent, order-dependent and infrastructure. Parry et al. [59] used a machine-learning approach to classify tests into two groups: order-dependent and non-order-dependent. They experimented with the test suites of 26 Python projects to evaluate whether static and dynamic features were more effective as flakiness predictors than previous feature sets.

B. Flaky tests in JavaScript

Hashemi et al. [13] investigated flaky test causes and the actions taken by developers to fix or mitigate flaky tests in JavaScript projects. The study identified concurrency, async wait, OS and network as the top four causes of test flakiness in JavaScript projects. Unlike previous empirical studies on Java [11], [14] and Python [12], which identify test order dependency as one of the key causes of flakiness, Hashemi et al. [13] found only very few flaky tests are caused by test order dependency. Similarly, Costa et al. [60] investigated flaky tests across five programming languages, including JavaScript. The study reported that *async wait*, *concurrency*, and *platform dependency* are the top flakiness causes in JavaScript.

Yost [15] leveraged stress testing and reordering test suites to uncover flaky tests in JavaScript programs. By subjecting the application to stress conditions and modifying the order of test suite execution, the aim was to reveal the hidden issues that might not surface under normal circumstances. This study differs from ours in that they reordered test suites 10 times and ran each test suite only once, as well as also trying to introduce machine stress for non-order-dependency flakiness.

FlakyFix [61] used large language models to predict flaky test fix categories. To predict fixed categories, a machine-learning model analysed test code and identified patterns that lead to flakiness. They used two code models, CodeBERT and UniXCoder, and pre-trained them in six programming languages, including JavaScript.

Current flaky test detection tools are mainly available for Java and Python [45]. Looking at previous reviews on tools that are used to detect flaky tests in JavaScript programs [45], [62], [63], we identified only two tools. King et al. [64] proposed a machine-learning approach to detect flaky tests. They model the flakiness problem as a disease with certain symptoms to facilitate the identification of its causes. The tool uses Bayesian networks to classify and predict flaky tests. NodeRacer [65] can help detect test flakiness caused by event races in Node.js applications. NodeRacer comprises three main phases, collecting information, inferring the relation between the callbacks and rerunning the application. It reruns the application several times to expose event race errors. NodeRacer specifically handles concurrency issues and does not address flaky tests directly.

CONCLUSION

In this paper, we systematically investigate test flakiness due to test order dependency in JavaScript projects using Jest. We developed a randomization approach, called `JS-TOD`, that targets test dependencies at three levels: tests, `describe` blocks and test suites. `JS-TOD` creates 10 randomised orders for each test item across the three categories, and for each new order, rerun each test suite 10 times. Our evaluation of 81 projects shows that order-dependent test flakiness is possible in JavaScript projects. Among 873 investigated test suites, we found 55 order-dependent test instances in 20 test suites across 10 projects. All of the order-dependent tests we found are caused by shared files (seven projects) or shared mocking state (three projects) between tests. The latter is a new cause of order-dependent tests that has not been reported previously in the context of other languages.

ACKNOWLEDGMENT

We acknowledge the support of the New Zealand SfTI National Science Challenge grant no. MAUX2004, the US National Science Foundation grant no. CCF-2145774 and CCF-2217696, and the Jarmon Innovation Fund.

REFERENCES

[1] M. Fowler, "Eradicating non-determinism in tests," https://martinfowler.com/articles/nonDeterminism.html, (Accessed on 10/02/2024).

[2] A. Sandhu, "How to fix flaky tests," 2015. [Online]. Available: https://tech.justeattakeaway.com/2015/03/30/how-to-fix-flaky-tests/

[3] J. Palmer, "Test flakiness – methods for identifying and dealing with flaky tests," 2019. [Online]. Available: https://engineering.atspotify.com/2019/11/18/test-flakiness-methods-for-identifying-and-dealing-with-flaky-tests/

[4] M. Machalica, A. Samylkin, M. Porth, and S. Chandra, "Predictive test selection," in *Proceedings of the International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019.

[5] W. Lam, A. Shi, R. Oei, S. Zhang, M. D. Ernst, and T. Xie, "Dependent-test-aware regression testing techniques," in *Proceedings of the ACM International Symposium on Software Testing and Analysis (ISSTA)*, 2020.

[6] S. Thorve, C. Sreshtha, and N. Meng, "An empirical study of flaky tests in android apps," in *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2018.

[7] J. Micco, "Google testing blog: Flaky tests at google and how we mitigate them," https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html, (Accessed on 25/03/2024).

[8] M. Machalica, W. Chmiel, S. Swierc, and R. Sakevych, "Probabilistic flakiness: How do you test your tests? - engineering at Meta," https://engineering.fb.com/2020/12/10/developer-tools/probabilistic-flakiness/, 2020, (Accessed on 12/03/2024).

[9] K. Herzig, M. Greiler, J. Czerwonka, and B. Murphy, "The art of testing less without sacrificing quality," in *Proceedings of the IEEE/ACM 37th IEEE International Conference on Software Engineering*, 2015.

[10] J. Raine, "Reducing flaky builds by 18x - the GitHub blog," https://github.blog/engineering/reducing-flaky-builds-by-18x/, 2021, (Accessed on 31/03/2024).

[11] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proceedings of the ACM International Symposium on Foundations of Software Engineering (FSE)*, 2014.

[12] M. Gruber, S. Lukasczyk, F. Kroiß, and G. Fraser, "An empirical study of flaky tests in python," in *Proceedings of the IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2021.

[13] N. Hashemi, A. Tahir, and S. Rasheed, "An empirical study of flaky tests in javascript," in *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2022.

[14] M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli, "Understanding flaky tests: The developer's perspective," in *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2019.

[15] G. A. Yost, "Finding flaky tests in javascript applications using stress and test suite reordering," Master's thesis, The University of Texas at Austin, 2023.

[16] M. Taleb, "JavaScript unit testing frameworks in 2024: A comparison · Raygun blog," https://raygun.com/blog/javascript-unit-testing-frameworks/, 2023, (Accessed on 09/02/2024).

[17] J. Community, "Issue #4386: Flakiness in tests with Jest," https://github.com/jestjs/jest/issues/4386, 2023, accessed: 2024-09-11.

[18] S. Zhang, D. Jalali, J. Wuttke, K. Muşlu, W. Lam, M. D. Ernst, and D. Notkin, "Empirically revisiting the test independence assumption," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, 2014, pp. 385–396.

[19] A. Shi, W. Lam, R. Oei, T. Xie, and D. Marinov, "iFixFlakies: A framework for automatically fixing order-dependent flaky tests," in *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2019.

[20] "react-testing-library," https://github.com/testing-library/react-testing-library/blob/main/src/__tests__/auto-cleanup-skip.js, (Accessed on 10/02/2024).

[21] "State of JavaScript 2023: Testing," https://2023.stateofjs.com/en-US/libraries/testing/, (Accessed on 09/02/2024).

[22] "Globals · Jest," https://jestjs.io/docs/api#describename-fn, (Accessed on 10/02/2024).

[23] "Option to run Jest in band and sort tests alphabetically," https://github.com/jestjs/jest/issues/4032#issuecomment-424427942, 2018, (Accessed on 10/02/2024).

[24] "Jest test schedular," https://github.com/jestjs/jest/blob/1a487c1803124c594bdd86ee24b5949025660bc3/packages/jest-cli/src/test_scheduler.js#L88-L96, (Accessed on 10/02/2024).

[25] "Setup and teardown · Jest," https://jestjs.io/docs/setup-teardown#order-of-execution, (Accessed on 10/02/2024).

[26] "Jest cli options · jest," https://jestjs.io/docs/cli, (Accessed on 10/02/2024).

[27] "Jest testsequencer," https://jestjs.io/docs/configuration#testsequencer-string, (Accessed on 10/02/2024).

[28] "Babel · babel," https://babel.dev/, (Accessed on 10/01/2024).

[29] "REST API endpoints for search - GitHub docs," https://docs.github.com/en/rest/search/search, (Accessed on 10/01/2024).

[30] S. Brisson, E. Noei, and K. Lyons, "We are family: analyzing communication in GitHub software repositories and their forks," in *Proceedings of the IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2020, pp. 59–69.

[31] "Jest v20.0.0," https://github.com/jestjs/jest/releases/tag/v20.0.0, (Accessed on 10/02/2024).

[32] Anonymous, "Detecting and evaluating order-dependent flaky tests in JavaScript - replication package," 2024. [Online]. Available: https://doi.org/10.5281/zenodo.13852085

[33] "Jasmine-matchers," https://github.com/JamieMason/Jasmine-Matchers, (Accessed on 10/02/2024).

[34] "timkindberg/jest-when: Jest support for mock argument-matched return values." https://github.com/timkindberg/jest-when, (Accessed on 10/01/2024).

[35] "tj/commander.js: node.js command-line interfaces made easy," https://github.com/tj/commander.js, (Accessed on 10/01/2024).

[36] "rbardini/jest-it-up: Automatically bump up global jest thresholds whenever coverage goes above them," https://github.com/rbardini/jest-it-up, (Accessed on 10/01/2024).

[37] "fetch-mock-jest," https://github.com/wheresrhys/fetch-mock-jest/releases/tag/v1.5.1, (Accessed on 10/02/2024).

[38] R. Wang, Y. Chen, and W. Lam, "iPFlakies: A framework for detecting and fixing Python order-dependent flaky tests," in *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, 2022.

[39] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "iDFlakies: A framework for detecting and partially classifying flaky tests," in *Proceedings of the IEEE Conference on Ssoftware Testing, Validation and Verification (ICST)*, 2019.

[40] J. Bell and G. Kaiser, "Unit test virtualization with VMVM," in *Proceedings of the 36th International Conference on Software Engineering*, 2014, pp. 550–561.

[41] "Building a JavaScript testing framework: Run all the tests in parallel," https://cpojer.net/posts/building-a-javascript-testing-framework#run-all-the-tests-in-parallel, (Accessed on 10/01/2024).

[42] A. Gyori, A. Shi, F. Hariri, and D. Marinov, "Reliable testing: detecting state-polluting tests to prevent test dependency," in *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*, 2015.

[43] M. Gruber and G. Fraser, "FlaPy: mining flaky python tests at scale," in *Proceedings of the IEEE/ACM International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, 2023.

[44] "The Jest object · Jest," https://jestjs.io/docs/jest-object#jestmockmodulename-factory-options, (Accessed on 10/01/2024).

[45] A. Tahir, S. Rasheed, J. Dietrich, N. Hashemi, and L. Zhang, "Test flakiness' causes, detection, impact and responses: A multivocal review," *Journal of Systems and Software*, vol. 206, p. 111837, 2023.

[46] A. Gambi, J. Bell, and A. Zeller, "Practical test dependency detection," in *Proceedings of IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2018.

[47] S. Rahman and A. Shi, "FlakeSync: Automatically repairing async flaky tests," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024.

[48] S. Rahman, A. Massey, W. Lam, A. Shi, and J. Bell, "Automatically reproducing timing-dependent flaky-test failures," in *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2024.

[49] Z. Dong, A. Tiwari, X. L. Yu, and A. Roychoudhury, "Flaky test detection in android via event order exploration," in *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2021.

[50] A. Shi, A. Gyori, O. Legunsen, and D. Marinov, "Detecting assumptions on deterministic implementations of non-deterministic specifications," in *2016 IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2016.

[51] S. Dutta, A. Shi, R. Choudhary, Z. Zhang, A. Jain, and S. Misailovic, "Detecting flaky tests in probabilistic and machine learning applications," in *Proceedings of the ACM International Symposium on Software Testing and Analysis (ISSTA)*, 2020.

[52] S. Dutta, A. Shi, and S. Misailovic, "FLEX: fixing flaky tests in machine learning projects by updating assertion bounds," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021.

[53] S. Rasheed, J. Dietrich, and A. Tahir, "On the effect of instrumentation on test flakiness," in *Proceedings of IEEE/ACM International Conference on Automation of Software Test (AST)*, 2023.

[54] J. Dietrich, S. Rasheed, and A. Tahir, "Flaky test sanitisation via on-the-fly assumption inference for tests with network dependencies," in *Proceedings of the IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2022.

[55] A. Wei, P. Yi, T. Xie, D. Marinov, and W. Lam, "Probabilistic and systematic coverage of consecutive test-method pairs for detecting order-dependent flaky tests," in *Tools and Algorithms for the Construction and Analysis of Systems: 27th International Conference (TACAS)*, 2021.

[56] C. Li, M. M. Khosravi, W. Lam, and A. Shi, "Systematically producing test orders to detect order-dependent flaky tests," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2023.

[57] C. Li, C. Zhu, W. Wang, and A. Shi, "Repairing order-dependent flaky tests via test generation," in *Proceedings of the International Conference on Software Engineering (ICSE)*, 2022.

[58] C. Li and A. Shi, "Evolution-aware detection of order-dependent flaky tests," in *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2022.

[59] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "Evaluating features for machine learning detection of order-and non-order-dependent flaky tests," in *Proceedings of the IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2022.

[60] K. Costa, R. Ferreira, G. Pinto, M. d'Amorim, and B. Miranda, "Test flakiness across programming languages," *IEEE Transactions on Software Engineering*, 2022.

[61] S. Fatima, H. Hemmati, and L. Briand, "FlakyFix: Using large language models for predicting flaky test fix categories and test code repair," *arXiv preprint arXiv:2307.00012*, 2024.

[62] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "A survey of flaky tests," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 1, 2021.

[63] B. Zolfaghari, R. M. Parizi, G. Srivastava, and Y. Hailemariam, "Root causing, detecting, and fixing flaky tests: State of the art and future roadmap," *Software: Practice and Experience*, vol. 51, no. 5, 2021.

[64] T. M. King, D. Santiago, J. Phillips, and P. J. Clarke, "Towards a bayesian network model for predicting flaky automated tests," in *Proceedings of the IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, 2018.

[65] A. T. Endo and A. Møller, "NodeRacer: Event race detection for node.js applications," in *Proceedings of the IEEE International Conference on Software Testing, Validation and Verification (ICST)*, 2020.